2110446 DATA SCIENCE AND DATA ENGINEERING

Take-Home Midterm Exam: Traffy Fondue

Computer Engineering, Chulalongkorn University

Worralop Srichainont

March 9, 2026

Contents

I Introduction **1**

1 Traffy Fondue **1**

2 Project Overview **1**

3 Resources Links **2**

II Data Preparation **3**

4 Dependencies **3**

5 Load Dataset **4**

6 Data Cleaning **4**
6.1 Regex Pattern 4
6.2 Defining Class 6
6.3 Data Cleaning 7

7 Data Splitting **8**

8 PyTorch DataLoader **9**
8.1 Dependencies 9
8.2 Text AutoTokenizer 10
8.3 PyTorch Dataset 11
8.4 PyTorch DataLoader 12

9 Exploratory Data Analysis (EDA) **14**
9.1 Class Count 14
9.1.1 Visualization 14
9.1.2 Insights 14
9.2 Label Cardinality 16
9.2.1 Overall Visualization 16
9.2.2 Categorical Visualization 16
9.2.3 Insights 17
9.3 Co-occurrence Heatmap 18
9.3.1 Visualization 18
9.3.2 Insights 18
9.4 Token Length Distribution 19
9.4.1 Visualization 19
9.4.2 Insights 20
9.5 Non-Labeled Data Insight 21
9.5.1 Example Complaints 21
9.5.2 Keywords 21
9.5.3 Insights 21
9.6 Labeled Data Insight 22

9.6.1 Keywords . . . 22
9.6.2 Insights . . . 22
9.7 Abnormal Data . . . 23
9.7.1 คณะกรรมการการพัฒนาเศรษฐกิจ . . . 23
9.7.2 กระทรวงการท่องเที่ยวและกีฬา . . . 24
9.7.3 การไฟฟ้านครหลวง **และสำนักงาน กสทช.** . . . 25
9.7.4 Insights . . . 25

III Model **26**

10 Positive Weights **26**

11 PhayaThaiBERT Model **27**
11.1 Initialization . . . 27
11.2 Training Step . . . 28
11.3 Validation Step . . . 29
11.4 Prediction Step . . . 29

12 Model Training **30**
12.1 Initialization . . . 30
12.1.1 DataModule . . . 30
12.1.2 PhayaThaiBERT Model . . . 30
12.1.3 Early Stopping . . . 30
12.1.4 Model Checkpoint . . . 31
12.1.5 Progress Bar . . . 31
12.2 PyTorchLightning Trainer . . . 31

13 Model Fine-Tuning **32**
13.1 Dynamic Probability Threshold . . . 32
13.2 Validation Dataset Prediction . . . 33

14 Validation Result **34**
14.1 Confusion Matrix . . . 34
14.2 Classification Report . . . 35

IV Result **36**

V Discussion **37**

15 Problem & Solution Analysis **37**
15.1 Problem . . . 37
15.2 Solution . . . 37

16 Future Improvement **37**

VI Conclusion **38**

Part I
Introduction

1 Traffy Fondue

Traffy Fondue is a digital platform developed by Thailand's National Electronics and Computer Technology (NECTEC) which allows residents to report infrastructure issues such as broken streetlights, garbage or flooding.

It is adopted by the Bangkok Metropolitan Administration (BMA) as a central digital platform for the residents to report their issue and uses Artificial Intelligence (AI) to automatically send to the correct government department. The residents can access the service via LINE messaging app and a website. After reporting the issues, users receive updates on the progress.

In March 2026, Bangkok Traffy Fondue has received more than 1,200,000 reports and more than 1,000,000 reports resolved. This shows that the system clearly improves efficiency, transparency, and citizen engagement of Bangkok government workflow.

2 Project Overview

This project focuses on building a pipeline to perform data preprocessing and implement machine learning model to classify the input report text according to government department below.

1. สำนักงานตำรวจแห่งชาติ
2. การรถไฟฟ้าขนส่งมวลชนแห่งประเทศไทย
3. สภาเด็กและเยาวชนกรุงเทพมหานคร
4. กรมควบคุมมลพิษ
5. กรมสรรพสามิต
6. การไฟฟ้านครหลวง
7. กรมทางหลวง
8. สำนักงานประกันสุขภาพแห่งชาติ
9. การประปานครหลวง
10. คณะกรรมการการพัฒนาเศรษฐกิจ
11. กระทรวงการท่องเที่ยวและกีฬา
12. สำนักงาน กสทช. ศูนย์รับแจ้งปัญหา 1200

Overall, this project uses **PyThaiNLP** for data preprocessing, **PhayaThaiBERT** for deep learning model and use **Macro F1 score** for evaluation.

3 Resources Links

Resource	**Link**
Kaggle Dataset	Link
Raw Train Dataset	Link
Raw Test Dataset	Link
Cleaned Train Dataset	Link
Cleaned Test Dataset	Link
Model Checkpoint	Link
Data Preprocessing Notebook	Link
EDA Notebook	Link
Model Notebook	Link

Table 1: Resources Link

Part II
Data Preparation

4 Dependencies

Since the Kaggle Notebook or Google Colab Notebook environment doesn't have PyThaiNLP and Iterative Stratification library, we need to install the library using `pip`.

Install Libraries

```
pip -q install pythainlp python-crfsuite
pip install -U iterative-stratification
```

Then import these libraries to perform the data preprocessing.

- `re` is for searching, matching and replace text in Python.
- `numpy` is for data array for data splitting.
- `pandas` is for tabular data manipulation.
- `tqdm` is for visualizing progress bar.
- `pythainlp` is for manipulating Thai text.
  - `normalize` is for reorder the Thai alphabet of every Thai text to the same standard.
  - `ThaiTextProcessor` is for creating the text processor object of PyThaiNLP.
- `MultilabelStratifiedShuffleSplit` is for applying stratification for multi-label data splitting.

Dependencies

```python
import re
import numpy as np
import pandas as pd

from tqdm import tqdm
from pythainlp.util import normalize
from pythainlp.phayathaibert import ThaiTextProcessor
from iterstrat.ml_stratifiers import MultilabelStratifiedShuffleSplit
```

5 Load Dataset

In the project, I have create the Traffy Fondue dataset in Kaggle which can be accessed by this link: Kaggle Dataset

After adding Kaggle dataset into the Kaggle notebook, load the dataset using the pandas command.

Load Dataset

```
TRAIN_FILE_PATH = "/kaggle/input/datasets/worralopsrichainont/2110446-dsde-midterm-dataset-cleaned/train.csv"
TEST_FILE_PATH = "/kaggle/input/datasets/worralopsrichainont/2110446-dsde-midterm-dataset-cleaned/test.csv"

df_train = pd.read_csv(TRAIN_FILE_PATH)
df_test = pd.read_csv(TEST_FILE_PATH)
```

6 Data Cleaning

6.1 Regex Pattern

To make the regex operation easier to debug, create a pattern constant for each category.

- **HTML Pattern** use to find and match any tags inside `<>` tags (e.g. `<body>`) in the text.
- **Phone Pattern** use to find and match Thai telephone number in the text.
  - **Thai Mobile Number**: Telephone number with 10 digits starts with 06 or 08 or 09
  - **Thai Landlines Number**: Telephone number with 9 digits starts with 02 or 03 or 04 or 09
  - **Thai Toll-free Number**: Telephone number with 10 digits starts with 1800 or 1900 or 1401
  - **Thai Hotlines**: 1555, 1129, 1125, 1137, 1644, 1669, 191, 199
- **Email Pattern** use to find and match any email in the text.
- **URL Pattern** use to find any URL in the text.
- **Zero Width Pattern** use to find any zero width characters.
- **Whitelist Character Pattern** use to filter only Thai characters, English characters digits, some punctuations and spaces.
- **Whitespace Pattern**: use to convert newline spaces and tab spaces to spaces.

Regex Pattern

```python
HTML_PATTERN = re.compile(r"<[^<>]*>")

PHONE_PATTERN = re.compile(
    r"(?<![\d/.:])(?:"
    r"0[689]\d[- ]?\d{3}[- ]?\d{4}|"
    r"02[- ]?\d{3}[- ]?\d{4}|"
    r"0[3457]\d[- ]?\d{3}[- ]?\d{3}|"
    r"(?:1800|1900|1401)[- ]?\d{3}[- ]?\d{3}|"
    r"1555|1129|1125|1137|1644|1669|191|199"
    r")(?![\d/.:])"
)

EMAIL_PATTERN = re.compile(r"[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+")

URL_PATTERN = re.compile(r"https?://[^\s\u0E01-\u0E5B]+|www\.[^\s\u0E01-\u0E5B]+")

ZERO_WIDTH_CHARS_PATTERN = re.compile(r"[\u200B\u200C\u200D\uFEFF]+")

PUNCTUATION_PATTERN = re.compile(r'([!"\'(),./:;?#-])\1+')

ALLOWED_PUNC = r"""!"'(),./:;?#-"""
WHITELIST_PATTERN = re.compile(rf"[^a-zA-Z0-9\u0E01-\u0E5B\s{re.escape(ALLOWED_PUNC)}฿]”)

WHITESPACE_PATTERN = re.compile(r"\s+")
```

6.2 Defining Class

The Thai text cleaning process is as follows

1. Convert all English characters to the lowercase and remove trailing spaces.
2. Remove HTML tags.
3. Remove telephone number.
4. Remove email.
5. Remove URL.
6. Remove zero width characters.
7. Remove all characters that is not Thai characters, English characters, digits, some punctuations and space.
8. Convert all tab space (`"\t"`) and newline space (`"\n"`) to space (`" "`).
9. Add space between `#` and its content (e.g. `#traffyfondue` → `# traffyfondue`)
10. Reorder all Thai characters of the text to the same standard.
11. Remove redundant characters (e.g. convert รอนานมากกกกกกกก to รอนานมาก)

Text Cleaner Class

```python
class TextCleanerBERT:
    THAI_TEXT_PROCESSOR = ThaiTextProcessor()

    @staticmethod
    def get_cleaned_thai_text(text):
        if not isinstance(text, str):
            return text

        text = text.lower().strip()
        text = TextCleanerBERT.remove_html(text)
        text = TextCleanerBERT.remove_telephone_number(text)
        text = TextCleanerBERT.remove_email(text)
        text = TextCleanerBERT.remove_url(text)
        text = TextCleanerBERT.remove_zero_width_chars(text)
        text = TextCleanerBERT.remove_non_whitelist_chars(text)
        text = TextCleanerBERT.clean_punctuation(text)
        text = TextCleanerBERT.pad_hashtags(text)

        text = normalize(text)
        text = TextCleanerBERT.remove_redundant_chars(text)
        text = TextCleanerBERT.clean_whitespace(text)
        return text

    @staticmethod
    def remove_html(text):
        return HTML_PATTERN.sub("", text)

    @staticmethod
    def remove_telephone_number(text):
```

```python
        return PHONE_PATTERN.sub("", text)

    @staticmethod
    def remove_email(text):
        return EMAIL_PATTERN.sub("", text)

    @staticmethod
    def remove_url(text):
        return URL_PATTERN.sub("", text)

    @staticmethod
    def remove_zero_width_chars(text):
        return ZERO_WIDTH_CHARS_PATTERN.sub("", text)

    @staticmethod
    def remove_non_whitelist_chars(text):
        return WHITELIST_PATTERN.sub(" ", text)

    @staticmethod
    def clean_punctuation(text):
        return PUNCTUATION_PATTERN.sub(r"\1", text)

    @staticmethod
    def pad_hashtags(text):
        return text.replace("#", " # ")

    @staticmethod
    def remove_redundant_chars(text):
        return TextCleanerBERT.THAI_TEXT_PROCESSOR.replace_rep_after(text)

    @staticmethod
    def clean_whitespace(text):
        text = WHITESPACE_PATTERN.sub(" ", text)
        return text.strip()
```

6.3 Data Cleaning

Do the following operations to clean the dataset

1. Remove all rows with missing value by using `dropna()`.
2. Apply text cleaning function of train dataset.
3. Apply text cleaning function of test dataset.

Data Cleaning

```python
df_train = df_train.dropna()

tqdm.pandas(desc="Cleaning Train Dataset")
df_train["comment"] = df_train["comment"].progress_apply(
    TextCleanerBERT.get_cleaned_thai_text
)
tqdm.pandas(desc="Cleaning Test Dataset")
df_test["comment"] = df_test["comment"].progress_apply(
    TextCleanerBERT.get_cleaned_thai_text
)
```

7 Data Splitting

Now, we are going to perform the splitting all data rows in `train.csv` into train dataset and validation dataset by using stratification to ensure the class ratio in both dataset.

All rows in the file `train.csv` has 304,284 rows and 14 columns (including `id`, `comment` and other 12 categorical columns)

- **Train Dataset** used for training the model which has 273,855 rows and 14 columns (90% of data)
- **Validation Dataset** used for tuning the model hyper-parameters which has 30,429 rows and 14 columns (10% of data)

Configuration

```
VALIDATION_DATASET_RATIO = 0.1
SEED = 42

TEXT_COL = "comment"

LABEL_COLS = [
    "สำนักงานตำรวจแห่งชาติ",
    "การรถไฟฟ้าขนส่งมวลชนแห่งประเทศไทย",
    "สภาเด็กและเยาวชนกรุงเทพมหานคร",
    "กรมควบคุมมลพิษ",
    "กรมสรรพสามิต",
    "การไฟฟ้านครหลวง",
    "กรมทางหลวง",
    "สำนักงานประกันสุขภาพแห่งชาติ",
    "การประปานครหลวง",
    "คณะกรรมการการพัฒนาเศรษฐกิจ",
    "กระทรวงการท่องเที่ยวและกีฬา",
    "สำนักงาน กสทช. ศูนย์รับแจ้งปัญหา 1200”,
]
```

First, Extract feature and label columns from the dataset.

Extract Feature and Label

```
X = df_train_all['comment'].to_numpy()
y = df_train_all[LABEL_COLS].to_numpy()
```

Then, split the dataset using the multi-label stratification.

Multi-label Stratification

```
# Create a multi-label dataset splitter
msss = MultilabelStratifiedShuffleSplit(
    n_splits=1,
    test_size=VALIDATION_DATASET_RATIO,
    random_state=SEED
)
# Begin dataset splitting
for train_index, val_index in msss.split(X, y):
    df_train = df_train_all.iloc[train_index].reset_index(drop=True)
    df_val = df_train_all.iloc[val_index].reset_index(drop=True)
```

8 PyTorch DataLoader

8.1 Dependencies

After splitting train dataset and validation dataset in `pandas`, we need to create PyTorch DataLoader to properly feed data as a batch to the PyTorch model.

However, the Kaggle Notebook or Google Colab Notebook environment doesn't have transformer model library PyTorchLightning framework, we need to install the framework using `pip`.

Install Libraries and Frameworks

```
pip install -U lightning
pip install -U transformer
```

Then import these libraries to create the PyTorch DataLoader.

- `torch` is for create and implement deep learning model.
- `pytorch_lightning` is a framework for organize the model code.
- `Dataset` is for converting pandas dataframe into PyTorch dataset.
- `DataLoader` is for creating a wrapper for PyTorch Dataset which can send data as a batch to the PyTorch deep learning model.
- `AutoTokenizer` is for encoding Thai text for PhayaThaiBERT model.

Dependencies

```
import torch
import pytorch_lightning as pl
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer
```

To ensure the consistency of the result, we need to set the seed for every PyTorch operation. These are also including model section.

Setup PyTorch Seed

```
pl.seed_everything(42, workers=True)
```

8.2 Text AutoTokenizer

Before feeding the text into the PhayaThaiBERT deep learning model, we need to tokenize them first. In this section, we will loading the PhayaThaiBERT pre-trained model for a tokenizer.

Tokenizer Configuration

```
MODEL_NAME = "clicknext/phayathaibert"
MAX_LENGTH = 128
TEXT_COL = "comment"
```

Now, we are going to implement the tokenizer from the PhayaThaiBERT pre-trained model. As for the maximum token length, we are choosing 128 tokens according to the EDA.

Thai Text Tokenizer

```
class ThaiTokenizer:
    # Load pre-trained PhayaThaiBERT tokenizer
    TOKENIZER = AutoTokenizer.from_pretrained(MODEL_NAME)

    @staticmethod
    def get_encoded_text(text):
        # Encode text by using the tokenizer
        encoded_text = ThaiTokenizer.TOKENIZER(
            str(text),
            max_length=MAX_LENGTH,
            padding="max_length",
            truncation=True,
            add_special_tokens=True,
            return_attention_mask=True,
            return_tensors="pt",
        )

        # Return an encoded text
        return encoded_text
```

8.3 PyTorch Dataset

After creating the PhayaThaiBERT tokenizer to encode the text, we need to create the PyTorch dataset for compatible with the PyTorch deep learning model.

- `__init__` is the constructor method which contains
  - `has_label` is `True` for train dataset and validation dataset but `False` for test dataset.
  - `texts` is the report text column as a numpy array.
  - `labels` is the label columns as a numpy array.
  - `n_rows` is the rows amount of the dataset.
- `__len__` returns the rows amount of the dataset.
- `__getitem__` return the encoded text by the tokenizer of the index in the dataset which return as a dict of `input_ids`, `attention_mask` and `labels`.

PyTorch Dataset

```python
class TraffyFondueDataset(Dataset):
    def __init__(self, df, has_label):
        # Store if the the dataset has label or not
        self.has_label = has_label

        # Separate text and label from a Pandas DataFrame
        self.texts = df[TEXT_COL].to_numpy()
        if self.has_label:
            self.labels = df[LABEL_COLS].to_numpy(dtype=np.float32)

        # Store rows amount of the dataset
        self.n_rows = df.shape[0]

    def __len__(self):
        # Return length of the dataset
        return self.n_rows

    def __getitem__(self, idx):
        # Encode the text using tokenizer
        encoded_text = ThaiTokenizer.get_encoded_text(self.texts[idx])

        # Return an item as a dict
        item = {
            "input_ids": encoded_text["input_ids"].squeeze(0),
            "attention_mask": encoded_text["attention_mask"].squeeze(0),
        }
        if self.has_label:
            item["labels"] = torch.tensor(self.labels[idx])

        return item
```

8.4 PyTorch DataLoader

After creating the PyTorch dataset, we still require PyTorch DataLoader to easily feed multiple dataset rows as a batch to the PyTorch deep learning model. Here, we are going to set batch size to 32 to make this DataLoader feeds 32 rows to the deep learning model at once.

DataLoader Configuration

```
BATCH_SIZE = 32
NUM_WORKERS = 2
```

Next, we are going to implement the PyTorch DataLoader as a class. In this part, it is the constructor method which initialize the batch size attribute, train dataset, validation dataset and test dataset.

PyTorch DataLoader Constructor

```
class TraffyFondueDataModule(pl.LightningDataModule):
    def __init__(self, df_train, df_val, df_test):
        super().__init__()
        self.batch_size = BATCH_SIZE
        self.num_workers = NUM_WORKERS

        self.df_train = df_train
        self.df_val = df_val
        self.df_test = df_test
```

Then, create the train PyTorch DataLoader from the train PyTorch Dataset. Make sure to set the `shuffle` to `True` for shuffling the order of data inside a batch to prevent a model to just memorize the data order pattern instead of actually understanding the context.

Train DataLoader

```
class TraffyFondueDataModule(pl.LightningDataModule):
    def train_dataloader(self):
        dataset_train = TraffyFondueDataset(self.df_train, True)

        return DataLoader(
            dataset_train,
            batch_size=self.batch_size,
            shuffle=True,
            num_workers=self.num_workers,
            pin_memory=True,
        )
```

Then, create the validation PyTorch DataLoader from the validation PyTorch Dataset. Set the `shuffle` to `False` because it is just for evaluation the model performance.

Validation DataLoader

```python
class TraffyFondueDataModule(pl.LightningDataModule):
    def val_dataloader(self):
        dataset_val = TraffyFondueDataset(self.df_val, True)

        return DataLoader(
            dataset_val,
            batch_size=self.batch_size,
            shuffle=False,
            num_workers=self.num_workers,
            pin_memory=True,
        )
```

Then, create the test PyTorch DataLoader from the test PyTorch Dataset. Set the `shuffle` to `False` because it is just for evaluation the model performance.

Test DataLoader

```python
class TraffyFondueDataModule(pl.LightningDataModule):
    def test_dataloader(self):
        dataset_test = TraffyFondueDataset(self.df_test, False)

        return DataLoader(
            dataset_test,
            batch_size=self.batch_size,
            shuffle=False,
            num_workers=self.num_workers,
            pin_memory=True,
        )
```

9 Exploratory Data Analysis (EDA)

In this section, we are going to focus about plotting and finding insights from the dataset before training a deep learning model.

9.1 Class Count

9.1.1 Visualization

In this part, we are going to visualize the amount of classes from the dataset which is shown below.

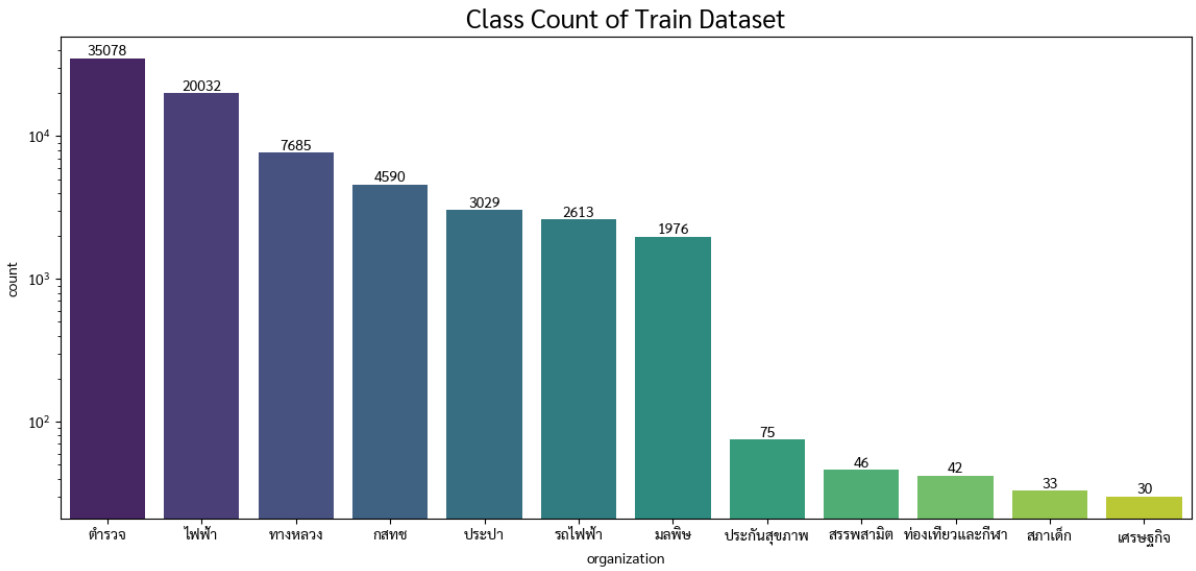

Figure 1: Class Count

9.1.2 Insights

The visualization shows that the dataset has extreme class imbalance and mostly are non-labeled data. Therefore, we need to customize the penalty weight on model validation process to make the model prioritize the minor classes too, not just the major classes.

The data table is in the next page →

Organization	**Count**	**Percentage**
Non-labeled	231,285	76.01%
สำนักงานตำรวจแห่งชาติ	35,078	11.53%
การไฟฟ้านครหลวง	20,032	6.58%
กรมทางหลวง	7,685	2.53%
สำนักงาน กสทช.	4,590	1.51%
การประปานครหลวง	3,029	1.00%
การรถไฟฟ้าขนส่งมวลชน	2,613	0.86%
กรมควบคุมมลพิษ	1,976	0.65%
สำนักงานประกันสุขภาพ	75	0.02%
กรมสรรพสามิต	46	0.02%
กระทรวงการท่องเที่ยวและกีฬา	42	0.01%
สภาเด็กและเยาวชน	33	0.01%
คณะกรรมการการพัฒนาเศรษฐกิจ	30	0.01%

Table 2: Class Count

9.2 Label Cardinality

9.2.1 Overall Visualization

In this part, we are going to visualize the amount of rows in each label cardinality. The label cardinality is the count of label of a single rows (e.g. a row with สำนักงานตำรวจ and การไฟฟ้านครหลวง has label cardinality equals to 2)

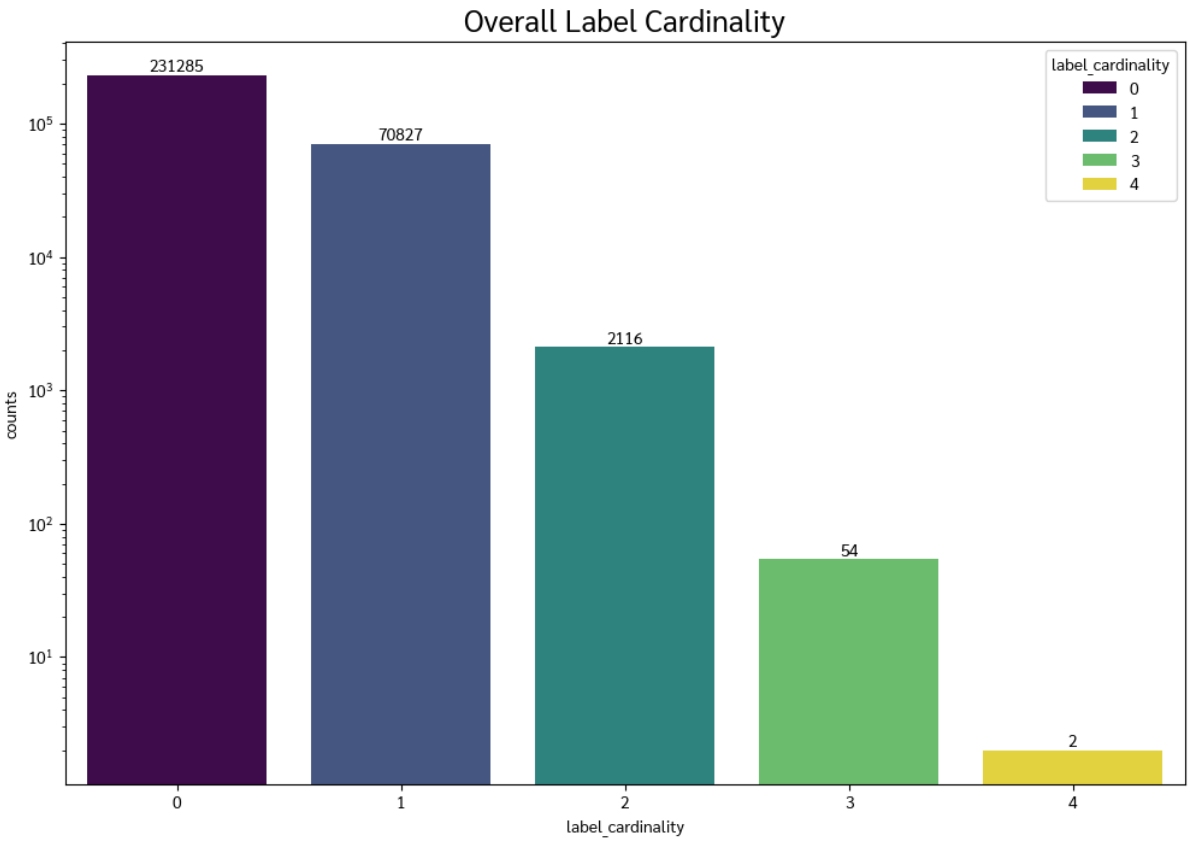

Figure 2: Overall Label Cardinality

9.2.2 Categorical Visualization

Next, we are going to visualize each label cardinality separated by classes to identify which category likely has the multi-label.

The visualization is in the next page →

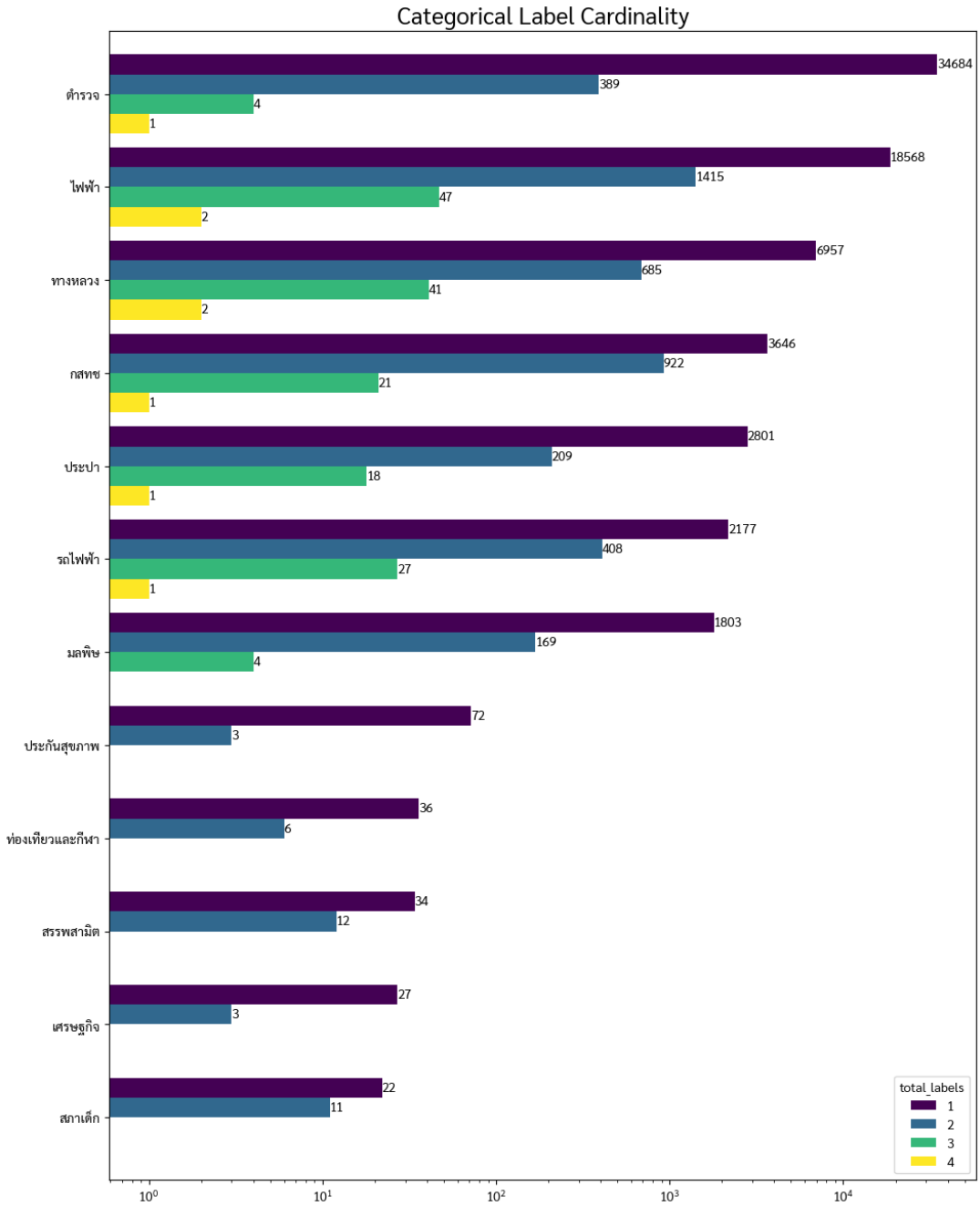

Figure 3: Categorical Label Cardinality

9.2.3 Insights

The visualization shows that the dataset is mostly non-labeled with 231,285 rows. In addition, the labeled data are mostly 1 label with 70,827 rows. This shows that while most complaints belong to one organization, the multi-label still exists so **BCEWithLogitsLoss** is the appropriate loss function for this task.

9.3 Co-occurrence Heatmap

9.3.1 Visualization

In this part, we are going to visualize the co-occurrence between the classes from the dataset which is shown below.

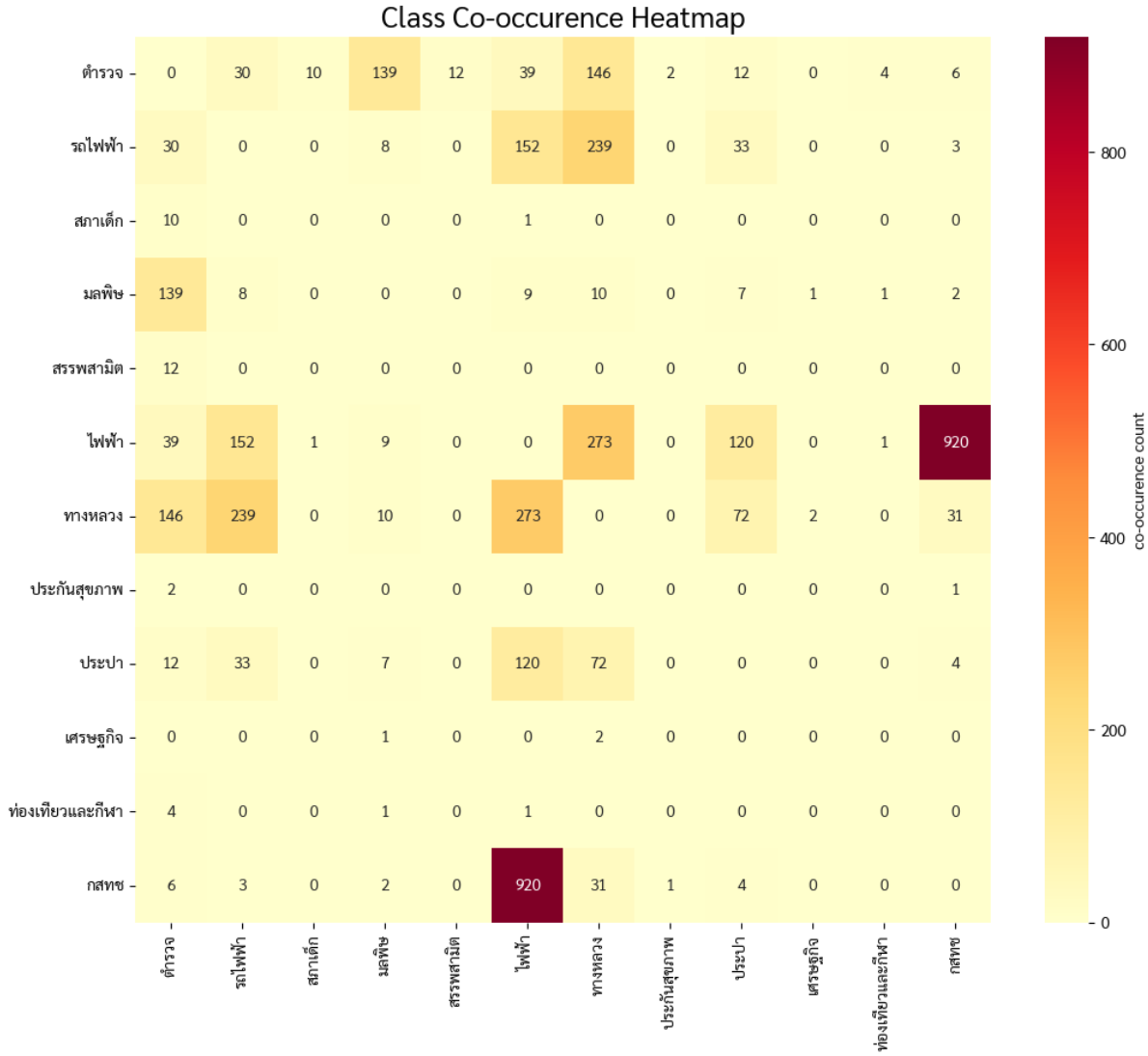

Figure 4: Co-occurrence Heatmap

9.3.2 Insights

The visualization shows that the การไฟฟ้านครหลวง organization and สำนักงาน กสทช has significant co-occurrence with 920 cases. These two organizations may have interesting shared parts of the complaints.

9.4 Token Length Distribution

9.4.1 Visualization

In this part, we are going to visualize the token length distribution of the complaints from the dataset which is shown below.

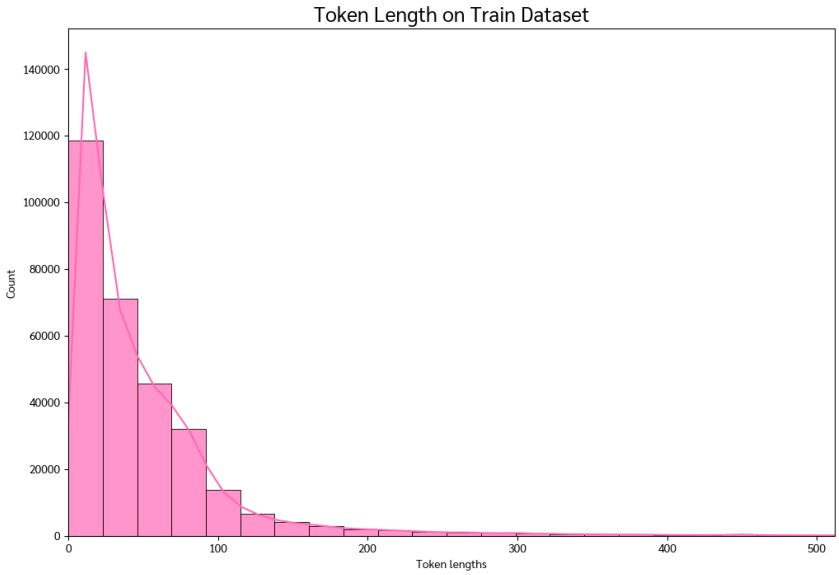

Figure 5: Train Dataset Token Length Distribution

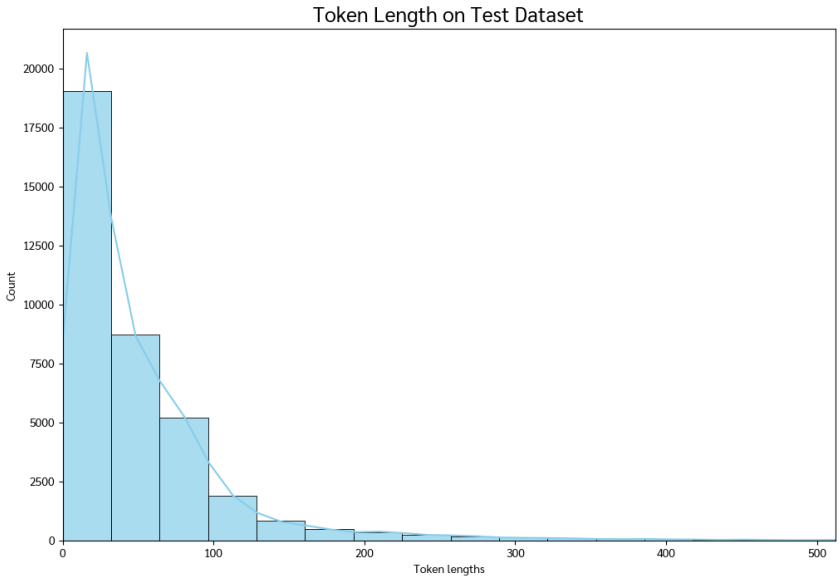

Figure 6: Test Dataset Token Length Distribution

9.4.2 Insights

The visualization shows that the token length distribution of both train dataset and test dataset have a similar distribution which is heavily right skew. According to the table below, we should set the maximum token length on the data preparation to 128 tokens.

Dataset	**Token Count**
Train Dataset	148 tokens
Test Dataset	153 tokens

Table 3: Token Count on 95th Percentile

9.5 Non-Labeled Data Insight

9.5.1 Example Complaints

Here are 5 samples of non-labeled data in the train dataset.

- ซอยชุ่มชื่น 4 ถนนอโศก-ดินแดง เขตดินแดง ตลอดทั้งซอยดังกล่าว เจ้าหน้าที่ไม่เข้าเก็บขนขยะ(ปกติเข้าเก็บวันอาทิตย์ วันอังคาร วันศุกร์) โดยรอบจัดเก็บวันศุกร์ที่ผ่านมาไม่เข้ามาเก็บ ทำให้เกิดปัญหาขยะตกค้างเป็นจำนวนมาก ผู้ร้องต้องการให้เจ้าหน้าที่นำรถขยะเล็กเข้ามาจัดเก็บภายในวันนี้(26/10/2567) เพราะไม่ต้องการรอให้เข้ามาเก็บในวันอาทิตย์ ขอให้เร่งดำเนินการตรวจสอบและเร่งแก้ไขโดยเร็ว
- แจ้งปัญหาฝุ่น ซอยวัดสุขใจ15 มีดินบนใหล่ทางตลอดแนว มีฝุ่นขาวฝุ่งกระจายทั้งวัน จนต้นไม้ข้างทางขาวเห็นได้ชัดเจน แจ้งเขตนำรถน้ำมาฉีดไล่ไหล่ทาง เพื่อลดฝุ่น ให้ชาวบ้านได้มีอากาศดีๆ ตอนนี้ดมกันแต่ฝุ่น
- ขยะในคลอง เกิดจากเด็กวัยรุ่น เข้ามาเล่นหุตซอลตรงสนาม มานั่งจับกลุ่ม ตรงริมคลอง กินเสร็จทิ้งลงคลอง บางคนก็ฉี่ลงคลอง ตัดควยทิ้งให้หมดอีพวกนี้ จัดการคุยกับเจ้าของด้วยนะครับ ว่าอย่าเปิดกิจการแล้วทำลายสภาพแวดล้อม
- มัสยิดละหมาดเสียงดังมากๆ ผมทำงานกะกลางคืน กลางวันก็ไม่เคยจะได้นอนเลย หอนวันละ5รอบ จะตะโกนออกไมค์เพื่อ คิดถึงคนอื่นบ้างที่เค้าก็มีสิทธิใช้ชีวิต ตื่นหรือนอนพักผ่อนตอนไหนก็ได้ ผมคนกะกลางคืนจะตายห่าละครับ ตื่นทุกๆ3-4ชม เพราะเสียเหี้ยเนี้ย ช่วยแก้สักทีได้มั้ย แจ้งกี่รอบแล้ว คนเวลามันหมดหนทางมันเครียดมากๆ ทำอะไรไม่คิดได้นะครับ เงินก็ต้องหา งานก็ต้องทำ เสียงเหี้ยนี้ก็มาทุกวัน ทั้งวัน ไม่ได้ผักผ่อน ก็ต้องตื่นไปทำงานอีกแล้ว
- ไอ้เย็ดเหี้ย เรื่องดีดีไม่คิดจะทำ ขยันทำเรื่องเดือดร้อนให้ชาวบ้าน แสบจมูก แสบคอ ขมคอ หงุดหงิด กลิ่นขยะมึงตั้งแต่เมื่อคืนจนตอนนี้แม่งก็ยังไม่ลด ไอ้พวกเย็ดแม่

9.5.2 Keywords

In this part, we are trying to perform clustering on 231,285 non-labeled data rows by using TF-IDF to tokenize words then using NMF algorithm for clustering. Here are the interesting group with keywords on the non-labeled data.

Group	Keywords
1	ทางเท้า จอด รถ รถจักรยานยนต์ ขับขี่ ขายของ ฟุตบาท ทางเดิน กีดขวาง
2	ขยะ ฝา ท่อ ทิ้ง ชำรุด รถ เหม็น ท่อระบายน้ำ แตก ล้น สกปรก ถังขยะ
3	ไฟ ดับ ป้าย ส่องสว่าง ชำรุด มืด เสา ติด ถนน ไฟถนน ไฟฟ้า ทางเท้า
4	หลุม ถนน บ่อ รถ อุบัติเหตุ ลึก ขัง พื้นถนน ฟุตบาท ขรุขระ ขนาดใหญ่
5	น้ำท่วม น้ำ ขัง ระดับ ความสูง ข้อเท้า ระบาย คน ท่อ

Table 4: Keywords on Non-labeled Data

9.5.3 Insights

According to the keyword clustering on 231,285 non-labeled data rows, they can be categorized into the pathway issue, garbage issue, streetlights issue, road hole issue and flood issue. Some of the non-labeled rows should be labeled to an organization instead of none, such as the complaints about streetlights should belong to การไฟฟ้านครหลวง or กรมทางหลวง However, some of the non-labeled have no context at all.

9.6 Labeled Data Insight

9.6.1 Keywords

In this part, we are performing c-TF-IDF (class based TF-IDF) on 72,999 labeled data rows to extract keywords from each class.

Organization	Keywords
สำนักงานตำรวจแห่งชาติ	กีดขวาง รถติด เลน ห้าม คัน จราจร อุบัติเหตุ รถยนต์
การรถไฟฟ้าขนส่งมวลชน	รถไฟฟ้า ก่อสร้าง สถานี สาย
สภาเด็กและเยาวชน	เข้าเรียน ขอทาน เยาวชน สอบถาม ซ้ำซ้อน อบรม สอดส่อง
กรมควบคุมมลพิษ	เผา ขยะ ควัน กลิ่น เหม็น ฝุ่น ส่งกลิ่น มลพิษ
กรมสรรพสามิต	ร้าน ขาย สุรา ใบอนุญาต บุหรี่ จำหน่าย ดนตรี เบียร์ เหล้า ร้านค้า หนีภาษี
การไฟฟ้านครหลวง	ดวง ส่องสว่าง ไฟฟ้า เสา ชำรุด เสาไฟฟ้า สายไฟ
กรมทางหลวง	สะพาน ทางเท้า ชำรุด หลุม ข้าม เส้น ส่องสว่าง อุบัติเหตุ เลน
สำนักงานประกันสุขภาพ	ส่งตัว คนไข้ โรงพยาบาล คลินิก การรักษา บัตรทอง รักษา สปสช
การประปานครหลวง	ท่อ ประปา วาง ขุด แตก น้ำประปา หลุม ไหล ซ่อม ท่อระบายน้ำ ฝา รั่ว
การพัฒนาเศรษฐกิจ	กระทรวง porn hub เว็บ ก่อสร้าง ดิจิตอล โป๊ แบน เศรษฐกิจ เว็บไซต์
การท่องเที่ยวและกีฬา	พลุ the and for traffywebsite วิจิตร that english
สำนักงาน กสทช.	สาย สายไฟ ห้อย สื่อสาร หย่อน ย้อย เสา ทางเท้า สัญญาณ ระโยงระยาง

Table 5: Keywords on Labeled Data

9.6.2 Insights

According to the keyword extraction on 72,999 labeled data rows, the keyword for most organizations are decent but there are 2 organizations which are การพัฒนาเศรษฐกิจ and การท่องเที่ยวและกีฬา that have unrelated keyword due to lack of data.

- คณะกรรมการการพัฒนาเศรษฐกิจ has unrelated keyword (e.g. porn hub)
- กระทรวงการท่องเที่ยวและกีฬา has English keyword (e.g. traffywebsite, english)

9.7 Abnormal Data

9.7.1 คณะกรรมการการพัฒนาเศรษฐกิจ

Here are 5 samples of train dataset rows of คณะกรรมการการพัฒนาเศรษฐกิจ

- ผมนายศุภกรณ์ อัจฉริยะพฤกษ์ไม่เห็นด้วยอย่างยิ่งที่กระทรวงดิจิตอลเพื่อเศรษฐกิจและสังคมแบนเว็บโป๊ porn hub ในประเทศไทยมานานกว่า3ปีแล้วโดยอ้างเรื่องศีลธรรมทั้งๆที่ทุกคนบนโลกใบนี้ก็เคยดูหนังโป๊แต่กลับมาแบนเว็บ porn hub แต่ไม่ยอมแบนเว็บโป๊อื่น เว็บ porn hub นี่ถือว่าปลอดภัยมากถ้าเทียบกับเว็บโป๊อื่นไม่ค่อยมีไวรัสไม่ค่อยมีมิจฉาชีพคอยดูดเงินเหมือนเว็บโป๊อื่น แต่เมื่อกระทรวงดิจิตอลเพื่อเศรษฐกิจและสังคมแบนแอป porn hub ทำให้เด็กวัยรุ่นไทยต้องไปดูหนัง โป๊ในเว็บอื่นและโดนดูดเงินโดยพวกหลอกหลวงพนันออนไลน์ จำนวนมากไม่ปลอดภัยเหมือนเว็บ porn hub ผมจึงขอฝากเขตหลักสี่ส่งเรื่องร้องเรียนของผมถึงกระทรวงดิจิตอลเพื่อเศรษฐกิจและสังคมในเลิกแบนเว็บโป๊ porn hub ในประเทศไทยและอนุญาติให้คนไทยทุกคนดูเว็บโป๊ porn hub ได้ตามเดิมเดี๋ยวนี้เลยครับ

- ผมนายอำนวย พุทธมีขอฝากเขตหลักสี่ส่งเรื่องร้องเรียนของผมถึงกระทรวงดิจิตอลเพื่อเศรษฐกิจและสังคมให้แบนแอปตั้งกระทู้ dek d ในประเทศไทยเพราะว่ามีการใช้คำหยาบคายและการบูลลี่มากมายในกระทู้นี้ไม่เหมาะสำหรับให้เด็กและเยาวชนเล่นแอปกระทู้ dek d ครับ

- นอกจากแบนกระทู้ dek d แล้วขอให้กระทรวงดิจิตอลเพื่อเศรษฐกิจและสังคมแบนเว็บไซต์วิกิพีเดียในประเทศไทยด้วยเนื่องจากข้อมูลในวิกิพีเดียมักเป็นข้อมูลปลอม ที่ไม่มีการคัดกรองจากแหล่งข้อมูลที่น่าเชื่อถือ มีการบิดเบือนข้อมูลจากความเป็นจริงมากมายซึ่งสร้างความเข้าใจผิดให้กับสังคมมาหลาย 10 ปีแล้ว

- ไซต์งานก่อสร้างขนาดใหญ่ กำลังก่อสร้างอยู่ในซอยที่มีแต่บ้านพักอาศัย แต่ขาดอุปกรณ์ป้องกัน สแสนกันเศษฝุ่นหรือสิ่งของที่อาจร่วงหล่นก็มีอยู่น้อย ทั้ง ๆ ที่ไซต์งานกับบ้านมีแค่คลองเล็กๆกั้น เลือกที่จะมาสร้างในบริเวณอยู่อาศัย ผู้รับเหมาควรมีจิตสำนึกต่อส่วนสังคมมากกว่านี้ มลพิษทางเสียง และอากาศ ควรสร้างให้ได้น้อยที่สุด ไซต์งานต้องมีสแลนหรือวัสดุป้องกันปกปิดมิดชิดทุกด้านเพื่อป้องกันไม่ให้เศษวัสดุอะไรปลิวลอยกระเด็นออกมา ยิ่งช่วงนี้กำลังจะเป็นช่วงฤดูฝน ควรต้องยิ่งป้องกันให้ดีกว่านี้

- มีอินฟลูเอนเซอร์ไทยไปนอนแผ่บนพื้นรถไฟฟ้าของประเทศญี่ปุ่นเพื่อทำคอนเทนท์ รวมถึงที่อื่นๆในญี่ปุ่น แล้วโพสต์ลงเฟสเอง แบบนี้คนญี่ปุ่นจะมองคนไทยยังไง เสียชื่อเสียมารยาทมาก แล้วเด็กดูเยอะ เดี๋ยวใครไปทำตามจะเสียหายกว่าเดิม ผิดกฎหมายข้อไหนมั้ยครับ ผิด พรบ.คอม มั้ยครับ

9.7.2 กระทรวงการท่องเที่ยวและกีฬา

Here are 5 samples of train dataset rows of กระทรวงการท่องเที่ยวและกีฬา

- parking warden whistles i realise that these guys are not employed for their intellect, but a blind man can see that madly whistling at cars that are moving, or cars that cannot move does not help an any way with the flow of traffic. all it does is deafen people nearby. for a taste, try the chaeng watthana road at the government complex at 5pm on any week day or seacon square on a weekend or public holiday. the noise is painful at less than 50 metres and still annoying at over 100m. and it is completely pointless. it achieves nothing. i ask that you please remove whistles from all security guards throughout the city. and it would be in society s best interests long term, if they were surgically altered so that they never reproduce. # traffywebsite # english
- it would be so good to have some speed bumps installed in this soi. the motorbikes and cars go so fast as if it is a race track. it is really dangerous for pedestrians. thank you # traffywebsite # english
- ปัญหา: บนสะพานพุทธยอดฟ้าและสะพานพระปกเกล้า พบมีการยิงพลุ เมื่อวันที่ 23 /11/67 ทำให้ประชาชนที่สัญจรผ่าน ได้มีการจอดรถเพื่อดูพลุเป็นเวลานาน ส่งผลให้การจราจรติดขัด ผู้ร้องจึงขอให้หากมีการจัดงานหรือมีการยิงพลุครั้งต่อไป ขอให้มีเจ้าหน้ามาอำนวยความสะดวกในเรื่องการจราจร เพื่อลดปัญหาการจราจรติดขัด ถนน: ตรีเพชร เขต: พระนคร
- ต่างด้าวและผู้มีอิทธิพล ตั้งวงดื่นสุราและมั่วสุม บนพื้นถนน และทางเท้า บริเวณวินรถจักรยานยนต์ ปากซอยราชปรารภ 8 (วินวัฒนวงศ์) ข้าง 7-11 เมื่อวันอังคารที่ 31 ธันวาคม 2567 กีดขวางรถดับเพลิง และรถพยาบาล ในการเข้าช่วยเหลือนักท่องเที่ยวและโรงแรมในซอย แบบนี้เกิดขึ้นบ่อยครั้งช่วงเทศกาลและเสาร์อาทิตย์ สร้างความเดือนร้อน ตลอดเวลาทั้งกลางวันและกลางคืน ดึกดื่น ช่วงเวลา ตี 1 ถึงตี 4 มีเสียงดังมาก อีกทั้งยังมีการลักลอบขโมยไฟต่อตรงอีกด้วย แจ้งเหตุไปยังสำนักงานเขตและตำรวจ หลายครั้งในช่วงเวลาหลายปี ก็ยังเป็นปัญหาเช่นเดิม
- ฟุตบาทด้านข้างกระทรวงคมนาคม ฝาปิดท่อปิดไม่สนิทแล้วเอาอะไรมาปิดชั่วคราวก็ไม่รู้ครับ ดูแล้วอันตรายมาก กลัวว่าขาจะหลุดลงไปในท่อ

9.7.3 การไฟฟ้านครหลวง และสำนักงาน กสทช.

Here are 5 samples of train dataset rows which are labeled as both การไฟฟ้านครหลวง and สำนักงาน กสทช.

- สายไฟสายสื่อสารไม่แน่ใจว่าเป็นสายอะไรครับแต่ทำไว้แล้วไม่เก็บให้เรียบร้อย อยู่หน้าปากซอยบางบอนสาม ซอย 1
- สายไฟ/สายสื่อสาร ขาดชำรุดหน้าปั้ม ปตท.ซอยบรม 97
- สายไฟห้อยตกลงมาจากเสาอันตรายคนเดินชนอาจเกิดอันตราย
- สายสื่อสารขาดและห้อยตกหน้าบ้านเป็นเวลานานแล้ว แจ้งกสทช.แต่ไม่มีการดำเนินการ เกรงจะเกิดอันตรายกับผู้สัญจร
- การไฟฟ้ามาขุดทางเท้าวางสายไฟแต่ไม่เก็บงานให้เรียบร้อยเดินลำบากระดับะนทางเท้าสูงๆต่ำๆ อันตรายไม่สวยงาม กทม. ถูกด่าแน่ๆ ฝากท่านประสานต่อด้วยครับ คนพลุกพลานย่านดังอาจจะเป็นข่าวได้

9.7.4 Insights

For 30 คณะกรรมการการพัฒนาเศรษฐกิจ data rows, there are mostly related to digital topic (such as Porn Hub website or Wikipedia website) and some rows are related to the foreign construction company.

For 42 กระทรวงการท่องเที่ยวและกีฬา data rows, there are significant more English complaints than the other organization. It is mostly about foreigner residents or traveller complaints about the issue in Bangkok especially transportation issues. Moreover, there are also Thai complaints which are mostly about the festival organized in Bangkok which are disturbing them as a local.

For 920 rows which are labeled as both การไฟฟ้านครหลวง and สำนักงาน กสทช. have the shared keyword สายไฟ and สายสื่อสาร which are appeared most of the complaints of this category.

However, due to a lack of data rows, the complaints sometimes seem unrelated to the organization and it is challenging to train a model that can predict those minor classes accurately.

Part III
Model

10 Positive Weights

According to the EDA, there are extreme class imbalance in the training dataset. Therefore, the penalty weight of each class should not be equal, otherwise the model would totally ignore the minor classes such as กระทรวงการท่องเที่ยวและกีฬา

We are calculating the weights (`pos_weight`) of each class for the model by this following formula

$$\texttt{pos_weight}_{\text{class}} = \min\left(\frac{\text{Number of Negative}_{\text{class}}}{\text{Number of Positive}_{\text{class}}}, 20\right)$$

Positive Weights Class

```python
class PositiveWeightCalculator:
    @staticmethod
    def get_pos_weight(df, max_pos_weight=20.0):
        # Get total rows of the DataFrame
        total_counts = df.shape[0]

        # Count positives in each class
        pos_counts = df[LABEL_COLS].sum(axis=0).values

        # Calculate positive weight
        pos_weight = (total_counts - pos_counts) / (pos_counts + 1e-5)
        pos_weight = torch.tensor(pos_weight, dtype=torch.float32)

        # Clamp the weights within the range
        pos_weight = torch.clamp(pos_weight, max=max_pos_weight)

        # Return positive weight
        return pos_weight
```

11 PhayaThaiBERT Model

In this section describes how to create a PhayaThaiBERT model for classification task by using PyTorchLightning framework to organize the model code as a class.

11.1 Initialization

First, write the constructor method (`__init__`) which contains

- `self.model` is pre-trained PhayaThaiBERT model. Set the output classes to 12.
- `self.loss_fn` is the BCEWithLogitsLoss function. Make sure to use the positive weights from the previous section.
- `self.macro_f1` is the macro F1 score as an evaluation metric of the model.

PhayaThaiBERT Model Initialization

```python
class TraffyFondueClassifier(pl.LightningModule):
    def __init__(self, pos_weight, n_classes=12, learning_rate=5e-5):
        super().__init__()

        # Download pre-trained PhayaThaiBERT model
        self.model = AutoModelForSequenceClassification.from_pretrained(
            MODEL_NAME, num_labels=n_classes
        )

        # Loss function
        self.loss_fn = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

        # Evaluation Metric
        self.macro_f1 = MultilabelF1Score(num_labels=n_classes, average="macro")

        # Ignore pos_weight on model logging
        self.save_hyperparameters(ignore=["pos_weights"])
```

Next, create an optimizer of the model. In this model, we are going to use Adam optimizer with dynamic learning rate adjustment. This will help the model to reach the local minimum on loss function quicker.

Adam Optimizer

```python
class TraffyFondueClassifier(pl.LightningModule):
    def configure_optimizers(self):
        # Use Adam with weight decay as optimizer
        optimizer = torch.optim.AdamW(
            self.parameters(), lr=self.hparams.learning_rate, weight_decay=0.01
        )
        return optimizer
```

11.2 Training Step

First, we need to define the forward pass of the model by following steps.

Model Forward Pass

```python
class TraffyFondueClassifier(pl.LightningModule):
    def forward(self, input_ids, attention_mask):
        # Pass tokenized text and attention through Transformer layers
        outputs = self.model(input_ids=input_ids, attention_mask=attention_mask)

        # Return logits score
        return outputs.logits
```

Next, define the training steps of the PhayaThaiBERT model which process as follows.

1. Get ID, attention mask and labels from a training data batch.
2. Utilize attention mask and labels in the model forward pass and got logits score.
3. Calculate the training loss by using BCEWithLogitsLoss function.
4. Logging the training loss

Model Training Step

```python
class TraffyFondueClassifier(pl.LightningModule):
    def training_step(self, batch, batch_idx):
        # Get information from a data batch
        input_ids = batch["input_ids"]
        attention_mask = batch["attention_mask"]
        labels = batch["labels"]

        # Calculate logits score and loss
        logits = self(input_ids, attention_mask)
        loss = self.loss_fn(logits, labels)

        # Log training results
        self.log(
            "train_loss", loss, on_step=True, on_epoch=True, prog_bar=True, logger=True
        )
        return loss
```

11.3 Validation Step

Define the validation steps of the PhayaThaiBERT model which process as follows.

1. Get ID, attention mask and labels from a validation data batch.
2. Utilize attention mask and labels in the model forward pass and got logits score.
3. Calculate the validation loss by using BCEWithLogitsLoss function.
4. Calculate macro F1 score by comparing the predicted value with the ground truth.
5. Logging both macro F1 score and validation loss.

Model Validation Step

```python
class TraffyFondueClassifier(pl.LightningModule):
    def validation_step(self, batch, batch_idx):
        # Get information from a data batch
        input_ids = batch["input_ids"]
        attention_mask = batch["attention_mask"]
        labels = batch["labels"]

        # Calculate logits score and loss
        logits = self(input_ids, attention_mask)
        loss = self.loss_fn(logits, labels)

        # Calculate the macro F1 score
        f1 = self.macro_f1(logits, labels)

        # Log validation results
        self.log("val_loss", loss, on_epoch=True, prog_bar=True, logger=True)
        self.log("val_macro_f1", f1, on_epoch=True, prog_bar=True,
        ↪  logger=True)
```

11.4 Prediction Step

Define the test dataset prediction steps of the PhayaThaiBERT model which process as follows.

1. Get ID and attention mask from a training data batch.
2. Utilize attention mask in the model forward pass and got logits score. Then, return it.

Model Prediction Step

```python
class TraffyFondueClassifier(pl.LightningModule):
    def predict_step(self, batch, batch_idx, dataloader_idx=0):
        # Get information from a data batch
        input_ids = batch["input_ids"]
        attention_mask = batch["attention_mask"]

        # Calculate logits score
        logits = self(input_ids, attention_mask)
        return logits
```

12 Model Training

In this section, we are going to focus on initialize and training the PhayaThaiBERT model.

12.1 Initialization

12.1.1 DataModule

First, initialize DataLoader to feed data batches into the model. The DataModule class is already defined on the Data Preparation part.

Initialize DataModule

```
data_module = TraffyFondueDataModule(df_train, df_val, df_test)
```

12.1.2 PhayaThaiBERT Model

Next, initialize the PhayaThaiBERT model by using the customized weights.

Initialize PhayaThaiBERT Model

```
# Calculate pos_weight for a model
pos_weight = PositiveWeightCalculator.get_pos_weight(df_train)

# Create a model
model = TraffyFondueClassifier(pos_weight)
```

12.1.3 Early Stopping

Then, initialize early stopping feature for the PyTorchLightning Trainer. Early stopping prevents the model from overfitting by immediately stop the model training process when the validation macro F1 score has not improved for 2 epochs

Initialize Early Stopping

```
early_stop = EarlyStopping(monitor="val_macro_f1", mode="max", patience=2)
```

12.1.4 Model Checkpoint

After that, initialize model checkpoint feature for storing the best model during training process measured by the validation macro F1 score in each epoch. The model in the last is not always the best model, so this feature can ensure that the model is always the best one.

Initialize Model Checkpoint

```
# Configuration
OUTPUT_PATH = "/kaggle/working/best_model/"
MODEL_FILENAME = "model-best-f1"

# Create a model checkpoint which saves the best model
checkpoint = ModelCheckpoint(
    monitor="val_macro_f1",
    mode="max",
    dirpath=OUTPUT_PATH,
    filename=MODEL_FILENAME,
    save_top_k=1,
)
```

12.1.5 Progress Bar

Finally, initialize model progress bar to only update the progress after 50 training data batches to prevent Kaggle from crashing.

Initialize Progress Bar

```
progress_bar = TQDMProgressBar(refresh_rate=50)
```

12.2 PyTorchLightning Trainer

After initializing all components in the previous section, we are going to build a PyTorch-Lightning Trainer.

Initialize PyTorchLightning Trainer

```
# Create a trainer to train a model
    trainer = pl.Trainer(
        accelerator="auto",
        devices="auto",
        precision="16-mixed",
        max_epochs=4,
        log_every_n_steps=10,
        val_check_interval=0.25,
        callbacks=[early_stop, checkpoint, progress_bar],
    )
```

Then, simply pass the data module and the PhayaThaiBERT model and train it.

Model Training

```
trainer.fit(model=model, datamodule=data_module)
```

13 Model Fine-Tuning

13.1 Dynamic Probability Threshold

After getting the logits score, we need to use Sigmoid function $\sigma(\text{logits})$ to convert to the probability. Then use the threshold to classify by using the following formula

$$\text{Predicted Class} = \begin{cases} \text{Negative Class} & \text{if } \sigma(\text{logits}) \leq t \\ \text{Positive Class} & \text{if } \sigma(\text{logits}) > t \end{cases}$$

However, fixing the probability threshold (t) at 0.50 is not the optimal choice, we need to dynamically adjust the probability threshold (t) for each class to maximize the macro F1 score by following these steps for each classes.

1. Filter only the prediction logits score column for current class from the dataset.
2. Apply Sigmoid function $\sigma(\text{logits})$ to convert to probability.
3. Iterate the probability threshold from $t = 0.05$ to $t = 0.95$
4. Lock the best probability threshold t for the current class.

Logits Converter Implementation

```python
class LogitsConverter:
    def __init__(self, n_classes):
        self.n_classes = n_classes
        self.thresholds = np.full(self.n_classes, 0.5)

    def tuning_all_thresholds(self, predictions, y_true):
        # Convert logits score to probability
        logits = torch.cat(predictions, dim=0)
        probs = torch.sigmoid(logits).numpy()

        # Iterate each class and calculate the best threshold
        for idx in range(self.n_classes):
            self.tuning_class_thresholds(probs[:, idx], y_true[:, idx], idx)

    def tuning_class_thresholds(self, probs_class, y_true_class, idx):
        best_thresh = 0.5
        best_f1 = 0.0

        # Iterate each threshold and find the best one
        for thresh in np.arange(0.05, 0.95, 0.01):
            # Predict by using the current threshold
            y_pred_class = (probs_class > thresh).astype(int)

            # Calculate F1 score from a prediction
            f1 = f1_score(y_true_class, y_pred_class, zero_division=0)

            # Save the best threshold for the class
            if f1 > best_f1:
                best_f1 = f1
                best_thresh = thresh

        # Update to class attributes
        self.thresholds[idx] = best_thresh
```

```python

    def get_all_binary(self, predictions):
        # Convert logits score to probability
        logits = torch.cat(predictions, dim=0)
        probs = torch.sigmoid(logits).numpy()

        # Create an empty predictions array
        n_rows = probs.shape[0]
        y_pred = np.zeros((n_rows, self.n_classes), dtype=int)

        # Iterate each class and get its predictions
        for idx in range(self.n_classes):
            y_pred[:, idx] = self.get_class_binary(probs[:, idx], idx)

        # Return all prediction
        return y_pred

    def get_class_binary(self, probs_class, idx):
        # Predict by using the tuned threshold
        y_pred_class = (probs_class > self.thresholds[idx]).astype(int)

        # Return class prediction
        return y_pred_class
```

13.2 Validation Dataset Prediction

After implementing the dynamic probability threshold for classification, we can just utilize those implemented methods.

First, get the validation logits score prediction from the trained model.

Validation Logits Score

```python
val_predictions = trainer.predict(
    model, dataloaders=data_module.val_dataloader(), ckpt_path="best"
)
```

Next, tuning the probability threshold and use them to convert logits score into actual class prediction

Tuning Probability Threshold & Predictions

```python
# Tuning prediction thresholds
logits_converter = LogitsConverter(len(LABEL_COLS))
logits_converter.tuning_all_thresholds(val_predictions, y_true)

# Get predictions as an array of 0 and 1
y_pred = logits_converter.get_all_binary(val_predictions)
```

Finally, get the ground truth label and perform an evaluation, the result of which are mentioned in the next section.

Ground Truth

```python
y_true = df_val[LABEL_COLS].values
```

14 Validation Result

14.1 Confusion Matrix

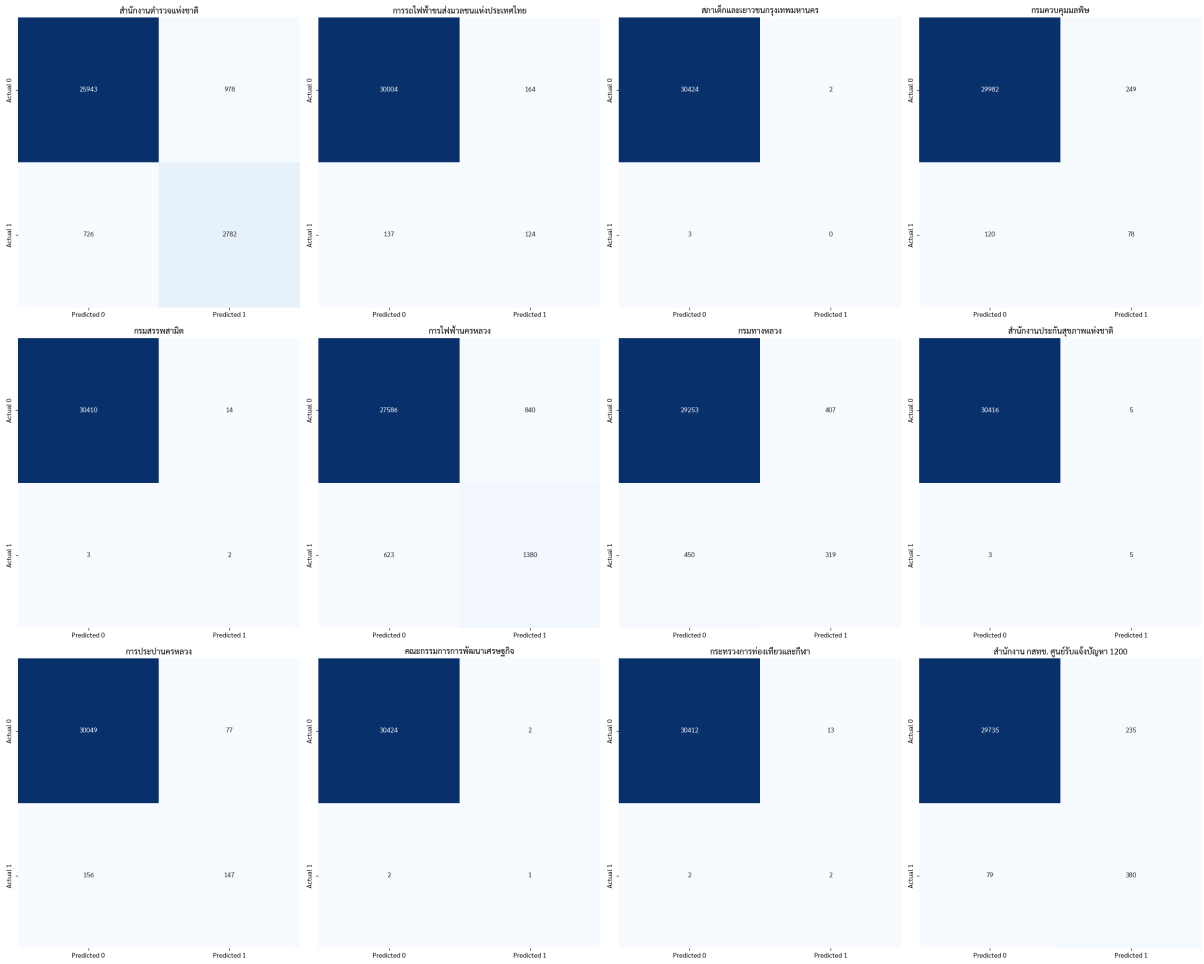

Figure 7: Validation Confusion Matrix

14.2 Classification Report

Organization	Precision	Recall	F1 Score	Support
สำนักงานตำรวจแห่งชาติ	0.74	0.79	0.77	3,508
การรถไฟฟ้าขนส่งมวลชน	0.43	0.48	0.45	201
สภาเด็กและเยาวชน	0.00	0.00	0.00	3
กรมควบคุมมลพิษ	0.24	0.39	0.30	198
กรมสรรพสามิต	0.12	0.40	0.19	5
การไฟฟ้านครหลวง	0.62	0.69	0.65	2,003
กรมทางหลวง	0.44	0.41	0.43	769
สำนักงานประกันสุขภาพ	0.50	0.62	0.56	8
การประปานครหลวง	0.66	0.49	0.56	303
การพัฒนาเศรษฐกิจ	0.33	0.33	0.33	3
การท่องเที่ยวและกีฬา	0.13	0.50	0.21	4
สำนักงาน กสทช.	0.62	0.83	0.71	459
Type	**Precision**	**Recall**	**F1 Score**	**Support**
Micro Average	0.64	0.69	0.66	7,524
Macro Average	0.40	0.49	0.43	7,524
Weighted Average	0.64	0.69	0.66	7,524
Samples Average	0.17	0.17	0.17	7,524

Table 6: Validation Classification Report

Part IV
Result

The submission got 0.36637 Macro F1 Score on the public leaderboard

#	Team	Members	Score	Entries	Last	Solution
76	6632200221_Worralop		0.36637	9	12d	

Figure 8: Public Leaderboard Result

The submission got 0.34254 Macro F1 Score on the private leaderboard

#	△	Team	Members	Score	Entries	Last	Solution
70	▲ 6	6632200221_Worralop		0.34254	9	12d	

Figure 9: Private Leaderboard Result

Part V
Discussion

15 Problem & Solution Analysis

15.1 Problem

According to the dataset, it has several problems during the deep learning model implementation including

- **Text Characters**: The raw dataset has lots of emojis and weird unicode characters which decreased the efficiency of the model and wasted the computational resources.
- **Non-Labeled Data**: About 76.01% of the dataset are the non-labeled dataset. Moreover, the topic of these data sometimes should be considered as labeled.
- **Extreme Class Imbalance**: The dataset has only few rows for สำนักงานประกันสุขภาพแห่งชาติ กรมสรรพสามิต กระทรวงการท่องเที่ยวและกีฬา สภาเด็กและเยาวชนกรุงเทพมหานคร and คณะกรรมการการพัฒนาเศรษฐกิจ (0.07% of dataset in total). This heavily affects the macro F1 score since it weights all classes equally.

15.2 Solution

With those mentioned problems, we did implement some features to fix them.

- **Text Cleaning**: All complaints text are thoroughly cleaned with punctuation, characters order, spaces and remove all emojis and weird unicode characters. This improves the efficiency of the model.
- **Positive Weight**: The positive weight is calculated to give more penalty to the model when it predicts the minor classes wrong. This makes the model to actually learn to predict those classes instead of dropping them which improves macro F1 score.

16 Future Improvement

After successfully implementing this project, the author has thought the possible future improvement as follows:

- **Data Filtering**: In the project, the author did not filter out some of the unrelated report, such as “test ka”, “by traffy staff report for test # traffywebsite # english”.
- **Postprocessing**: The author has already performed the simple postprocessing by finding and matching a keyword then force change the prediction label. However, this method significantly decreases the score from 0.34254 to 0.23804. There might be more effective postprocessing method to improve the score.
- **Non-labeled Data**: As mentioned that the non-labeled is the majority of the dataset (76.01%), training a model with all of them might not be a good idea. There might be more effective way to deal with this.

Part VI
Conclusion

In summary, the objective of this project is to design the data pipeline and implement a deep learning model to classify complaints text from Bangkok Traffy Fondue into the correct responsible government department.

For the data cleaning, this project uses Python pandas to load raw CSV file as a DataFrame, then uses Python regex to find a pattern to remove or replace, and then uses PyThaiNLP to normalize and encode the Thai text.

For the data pipeline, this project split the dataset into training dataset and validation dataset in 90:10 ratio by using iterative stratification method to ensure the equality of class ratio in both dataset. Next, we implement PyTorch DataModule to easily feed the data into a deep learning model as a batch.

For the deep learning model, this project uses PhayaThaiBERT model for classification. We load the pre-trained model, set BCEWithLogitsLoss as a loss function and use Macro F1 score as an evaluation metric. In addition, we also use Adam optimizer with dynamic learning rate.

For the fine-tuning process, this project dynamically adjust the probability threshold to convert logits score to class prediction for improving the validation macro F1 score.

According to the validation result and submission result in Kaggle competition, this model reaches 0.36637 Macro F1 Score on the public leaderboard and 0.34254 Macro F1 Score on the private leaderboard which indicates that the model is still not performing well on the minor classes, but performing satisfactorily on the major classes. However, this model still needs improvement in the future as discussed in the discussion part.